

Exercice 1 :

1. Créer une application Node.js appelée : employee-json-api
2. Installer les dépendances nécessaires :
 - express
 - nodemon
3. Créer un fichier employees.js contenant le tableau suivant :

```
const employees = [  
  {  
    id: 1,  
    matricule: "EMP001",  
    nom: "Ahmed",  
    prenom: "Ali",  
    email: "ahmed@gmail.com",  
    salaire: 7000  
  },  
  {  
    id: 2,  
    matricule: "EMP002",  
    nom: "Sara",  
    prenom: "Karimi",  
    email: "sara@gmail.com",  
    salaire: 8500  
  }  
]  
  
module.exports = employees
```

4. Créer un serveur Express utilisant express.json().
5. Créer les routes suivantes utilisant le tableau JSON :

Méthode	Route	Description
GET	/employees	Retourner tous les employés
GET	/employees/:id	Retourner un employé par ID
POST	/employees	Ajouter un employé
PUT	/employees/:id	Modifier un employé
DELETE	/employees/:id	Supprimer un employé

Exercice 2 :

1. Créer une application appelée : employee-management-api
2. Créer un système d'authentification avec :

- register
- login
- password hash (bcrypt)
- token JWT (expiresIn = 1h)

3. Créer un middleware authMiddleware.js qui :

- vérifie le token JWT
- bloque l'accès si token absent ou invalide
- ajoute req.user

4. Créer un modèle User avec :

- email (required, unique)
- password (required)

5. Créer un modèle Employee avec :

- matricule (String, required, unique)
- nom (String, required)
- prenom (String, required)
- email (String, required, unique)
- dateEmbauche (Date, required)
- dateNaissance (Date, required)
- salaire (Number, required, min: 0)

NB. Toutes les routes employés doivent être protégées par JWT.

6. Créer une fonction createEmployee :

- reçoit req.body
- crée un employé
- retourne l'employé créé

7. Créer une fonction getAllEmployees :

- retourne tous les employés

8. Créer une fonction `getEmployeeById` :

- récupère un employé par ID
- retourne 404 si introuvable

9. Créer une fonction `updateEmployee` :

- met à jour matricule, nom, prenom, email, dateEmbauche, dateNaissance, salaire

10. Créer une fonction `deleteEmployee` :

- supprime un employé

11. Créer les routes suivantes :

Auth :

- `POST /auth/register`
- `POST /auth/login`

Employees (protégées) :

- `POST /employees`
- `GET /employees`
- `GET /employees/:id`
- `PUT /employees/:id`
- `DELETE /employees/:id`

12. Créer un `server.js` qui :

- connecte MongoDB
- configure dotenv
- utilise `express.json()`
- monte routes auth + employees
- démarre serveur

13. Créer un Dockerfile qui :

- utilise `node:18-alpine`
- installe dépendances
- copie le code
- expose port 3000
- lance `server.js`

14. Compléter le tableau suivant en donnant les commandes Docker correspondantes :

Opération demandée	Commande Docker
--------------------	-----------------

Créer une image Docker à partir du projet avec le nom employee-api	
Créer un container nommé container1 à partir de l'image employee-api	
Démarrer le container container1	
Créer un deuxième container nommé container2	
Arrêter le premier container container1	
Supprimer le container arrêté container1	
Afficher la liste des containers actifs uniquement	
Afficher la liste de tous les containers	
Afficher la liste des images Docker	
Supprimer l'image employee-api	